

多光子 Fock 态跃迁振幅的一种递归编程实现

原文: Expressive Power and Limitations of Multi-photon Quantum Neural Networks

arxiv: 2608.01365v1

2026 年 10 月 9 日

本文讨论一个 m 模无源线性光学网络中, n 光子 Fock 态之间的跃迁振幅

$$\langle t|U|s\rangle$$

如何直接由产生算符的变换得到其表达式, 并进一步得到递推计算方法。

全文采用如下单光子变换约定:

$$u_{ij} = \langle j|U|i\rangle$$

即 u_{ij} 表示一个光子从输入模式 i 到输出模式 j 的单光子跃迁振幅。

1 多光子 Fock 态的跃迁振幅计算

设输入和输出占据数分别为

$$|s\rangle = |s_1, \dots, s_m\rangle, \quad |t\rangle = |t_1, \dots, t_m\rangle,$$

其中

$$\sum_{i=1}^m s_i = \sum_{j=1}^m t_j = n.$$

对应的归一化 Fock 态为

$$|s\rangle = \prod_{i=1}^m \frac{(a_i^\dagger)^{s_i}}{\sqrt{s_i!}} |0\rangle, \quad |t\rangle = \prod_{j=1}^m \frac{(a_j^\dagger)^{t_j}}{\sqrt{t_j!}} |0\rangle. \quad (1)$$

无源线性光学网络满足

$$U a_i^\dagger U^\dagger = \sum_{j=1}^m u_{ij} a_j^\dagger \quad (2)$$

以及 (左侧是输入端真空态, 右侧是输出端真空态)

$$U |0\rangle = |0\rangle. \quad (3)$$

其中 $u_{ij} = \langle j|U|i\rangle$ 。

由式 (1),

$$U|s\rangle = \frac{1}{\sqrt{\prod_i s_i!}} \prod_{i=1}^m U(a_i^\dagger)^{s_i} |0\rangle.$$

由于

$$U(a_i^\dagger)^{s_i} = (U a_i^\dagger U^\dagger)^{s_i} U,$$

并利用 $U|0\rangle = |0\rangle$, 得到

$$U|s\rangle = \frac{1}{\sqrt{\prod_i s_i!}} \prod_{i=1}^m \left(U a_i^\dagger U^\dagger \right)^{s_i} |0\rangle.$$

代入式 (2):

$$U|s\rangle = \frac{1}{\sqrt{\prod_i s_i!}} \prod_{i=1}^m \left(\sum_{j=1}^m u_{ij} a_j^\dagger \right)^{s_i} |0\rangle. \quad (4)$$

于是

$$\langle t|U|s\rangle = \frac{1}{\sqrt{\prod_i s_i! \prod_j t_j!}} \langle 0| \left(\prod_{j=1}^m a_j^{t_j} \right) \prod_{i=1}^m \left(\sum_{k=1}^m u_{ik} a_k^\dagger \right)^{s_i} |0\rangle. \quad (5)$$

由于不同模式的玻色子产生算符满足

$$[a_i^\dagger, a_j^\dagger] = 0,$$

因此产生算符内部重排不产生额外的负号或相位。为使 (5) 式不为零, 最终必须恰好产生

$$t_1, t_2, \dots, t_m$$

个分别位于各输出模式的光子。

把 $a_k^\dagger$ 部分的算符展开, 一共有 n 个括号, 因此每一项都是 n 个 u 相加。我们约定把这个式子第一个括号贡献的 u 振幅写在 n 个 u 的第一个位置, 依此类推。因此对于任何一项, 前 s_1 个元素的第一下标都是 1, 接下来的 s_2 个元素的第一下标都是 2, 这样的一项若能满足要求, 只需要填充的第二下标满足上面的模式数, 因此符合条件的项数为

$$\frac{n!}{\prod_{j=1}^m t_j!}$$

对其中任意一项, 再考虑产生湮灭算符

$$\langle 0| \left(\prod_{j=1}^m a_j^{t_j} \right) \left(\prod_{j=1}^m (a_j^\dagger)^{t_j} \right) |0\rangle = \prod_{j=1}^m t_j!$$

将此视为上面有效的一项的重复次数, 则一共有 $n!$ 项符合要求。

现定义 $n \times n$ 的 $U_{s,t}$ 矩阵:

$$U_{s,t} = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1m} \\ u_{21} & u_{22} & \cdots & u_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ u_{m1} & u_{m2} & \cdots & u_{mm} \end{pmatrix}$$

其中 u_{ij} 是一个 $s_i \times t_j$ 行的子矩阵。我们希望证明：

$$\langle t | U | s \rangle = \frac{\text{Per}(U_{s,t})}{\sqrt{\prod_{i=1}^m s_i! \prod_{j=1}^m t_j!}}. \quad (6)$$

其中积和式（**Permanent**，也称永久式）定义如下，实际上就是行列式中把正负交错的符号全换成正号：

$$\text{Per}(A) = \sum_{\sigma \in S_n} \prod_{r=1}^n A_{r, \sigma(r)}. \quad (7)$$

可以发现其中的全排列恰有 $n!$ 项，其中的每一项恰为 n 个 u 的联乘，我们按如下方式约定：第一行的 u 元素写在第一位，第二行的 u 元素写在第二位，以此类推，最终得到的前 s_1 个元素的第一下标都是 1，接下来的 s_2 个元素的第一下标都是 2，.....，这和前述定义是一致的。并且根据其定义，每行每列都有且仅有一个元素被选中，因此输入端口和输出端口的个数必然是正确的，也就是说，前面的

$$\frac{n!}{\prod_{j=1}^m t_j!}$$

项都在这个定义中包含，只需要证明重复次数也与上面相同即可。

根据上述约定，若要保持该项的形式不变，每一行的元素只可能在同行的 u_{ij} 矩阵块内部重新选取。以第一列子矩阵 $u_{11}, u_{21}, \dots, u_{m1}$ 为例，其中有 t_1 个元素被选中，给这 t_1 个元素分配“在哪一列”的问题，就是 t_1 个元素的全排列 $t_1!$ 种。考虑全部的 m 列子矩阵，一共重复的次数为

$$\prod_{j=1}^m t_j!$$

也与上面相同，证毕。

不过，直接按照式 (6) 计算需要遍历永久函数中的 $n!$ 个排列。或者说，对于一般 $n \times n$ 矩阵的 Perm，其计算复杂度是 $O(n \cdot n!)$ ，随着光子数 n 规模的增大，经典计算机是无法高效处理的。一个“有效”些的算法是 Ryser 算法，但其时间复杂度仍然在 $O(n \cdot 2^n)$ ，属指数增长。事实上，Gaussian boson 采样展现的量子优势正是源于此。

不过，QNN 研究中经典模拟还是常用到上述多光子 Fock 态跃迁振幅的。针对 Fock 态的具体结构，以及考虑到实验研究中多为模式 m 固定（芯片结构不变）情形下增加光子数 n ，有文章提供了下述算法能实现 m 固定条件下关于 n 的多项式复杂度。介绍算法前，我们先将上述结果改写成递推形式。

2 跃迁振幅递推式

任选一个满足

$$s_i > 0$$

的输入模式 i 。

定义从输入占据数中去掉一个模式 i 光子的状态

$$\boxed{s_i^- = (s_1, \dots, s_i - 1, \dots, s_m).} \quad (8)$$

由 Fock 态定义,

$$|s\rangle = \frac{1}{\sqrt{s_i}} a_i^\dagger |s_i^-\rangle. \quad (9)$$

因此

$$\begin{aligned} \langle t|U|s\rangle &= \frac{1}{\sqrt{s_i}} \langle t|U a_i^\dagger |s_i^-\rangle \\ &= \frac{1}{\sqrt{s_i}} \langle t|U a_i^\dagger U^\dagger U |s_i^-\rangle \\ &= \frac{1}{\sqrt{s_i}} \sum_{j=1}^m u_{ij} \langle t|a_j^\dagger U |s_i^-\rangle. \end{aligned} \quad (10)$$

对于输出模式 j , 若

$$t_j > 0,$$

定义

$$\boxed{t_j^- = (t_1, \dots, t_j - 1, \dots, t_m).} \quad (11)$$

由

$$a_j |t\rangle = \sqrt{t_j} |t_j^-\rangle$$

取厄米共轭得到

$$\boxed{\langle t|a_j^\dagger = \sqrt{t_j} \langle t_j^-|.} \quad (12)$$

若 $t_j = 0$, 则

$$\langle t|a_j^\dagger = 0,$$

因此式 (10) 中只有 $t_j > 0$ 的项有贡献。于是

$$\begin{aligned} \langle t|U|s\rangle &= \frac{1}{\sqrt{s_i}} \sum_{j:t_j>0} u_{ij} \sqrt{t_j} \langle t_j^-|U|s_i^-\rangle \\ &= \boxed{\sum_{j:t_j>0} u_{ij} \sqrt{\frac{t_j}{s_i}} \langle t_j^-|U|s_i^-\rangle}. \end{aligned} \quad (13)$$

因此得到多光子跃迁振幅的递推关系

$$\boxed{\langle t|U|s\rangle = \sum_{j:t_j>0} u_{ij} \sqrt{\frac{t_j}{s_i}} \langle t_j^-|U|s_i^-\rangle, \quad s_i > 0.} \quad (14)$$

递推一次以后, 输入和输出的总光子数都从 n 变为 $n - 1$ 。最终到达真空态:

$$\boxed{\langle 0|U|0\rangle = 1.} \quad (15)$$

式 (14) 因此可以以式 (15) 为边界条件递归计算任意多光子跃迁振幅。

理论上, 输入模式 i 可以选择任意满足 $s_i > 0$ 的模式。为了让程序中的递归路径确定, 可以固定选择最小的满足 $s_i > 0$ 的 i 。程序实现时, 使用记忆化递归, 避免相同的子问题被重复计算, 每个子问题最多计算一次。

3 递归程序及其分析

3.1 完整代码

```
import numpy as np
from functools import lru_cache

def amplitude(U, s, t):
    """
    Calculate the bosonic Fock-state transition amplitude

     $\langle t|U|s\rangle$ 

    Convention:
     $u_{ij} = \langle j|U|i\rangle$ 

    Recursive relation:

     $\langle t|U|s\rangle =$ 
     $\sum_{\{j:t_j>0\}}$ 
     $u_{ij} * \sqrt{t_j / s_i}$ 
     $* \langle t_j^-|U|s_i^- \rangle$ 
    """

    U = np.asarray(U, dtype=complex)

    s = tuple(int(x) for x in s)
    t = tuple(int(x) for x in t)

    m = len(s)

    if U.shape != (m, m):
        raise ValueError("U must be an m x m matrix.")

    if len(t) != m:
        raise ValueError("s and t must have the same length.")

    if any(x < 0 for x in s):
        raise ValueError(
            "Occupation numbers in s must be non-negative."
        )

    if any(x < 0 for x in t):
        raise ValueError(
```

```

"Occupation numbers in t must be non-negative."
)

# Photon-number conservation
if sum(s) != sum(t):
return 0.0 + 0.0j

@lru_cache(maxsize=None)
def AMP(s, t):

# Vacuum boundary condition
if sum(s) == 0:
return 1.0 + 0.0j

# Choose the smallest occupied input mode i
i = next(k for k in range(m) if s[k] > 0)

# Remove one photon from input mode i
s_minus = list(s)
s_minus[i] -= 1
s_minus = tuple(s_minus)

total = 0.0 + 0.0j

# Enumerate all possible output modes j
for j in range(m):

if t[j] == 0:
continue

# Remove one photon from output mode j
t_minus = list(t)
t_minus[j] -= 1
t_minus = tuple(t_minus)

total += (
U[i, j]
* np.sqrt(t[j] / s[i])
* AMP(s_minus, t_minus)
)

return total

return AMP(s, t)

```

3.2 代码分析

程序中的

```
AMP(s, t)
```

表示

$$\langle t | U | s \rangle.$$

程序首先检查光子数守恒:

$$\sum_i s_i = \sum_j t_j.$$

如果不满足, 则直接返回零, 因为无源线性光学网络不能改变总光子数。

递归终止条件为

```
if sum(s) == 0:
    return 1.0 + 0.0j
```

对应

$$\langle 0 | U | 0 \rangle = 1.$$

然后选择

```
i = next(k for k in range(m) if s[k] > 0)
```

即选择第一个满足

$$s_i > 0$$

的输入模式。

接下来

```
s_minus[i] -= 1
```

实现

$$s \rightarrow s_i^-.$$

程序随后遍历

```
for j in range(m):
    if t[j] == 0:
        continue
```

对应递推式中的

$$\sum_{j:t_j>0}.$$

对于每个允许的 j ,

```
t_minus[j] -= 1
```

实现

$$t \rightarrow t_j^-.$$

最后的核心语句

```
total += (
    U[i, j]
    * np.sqrt(t[j] / s[i])
    * AMP(s_minus, t_minus)
)
```

逐项对应

$$u_{ij} \sqrt{\frac{t_j}{s_i}} \langle t_j^- | U | s_i^- \rangle.$$

因此整个程序直接实现式 (14):

$$\langle t | U | s \rangle = \sum_{j: t_j > 0} u_{ij} \sqrt{\frac{t_j}{s_i}} \langle t_j^- | U | s_i^- \rangle.$$

程序使用

```
@lru_cache(maxsize=None)
```

保存已经计算过的状态 (s, t) 。

如果不同的递归路径最终产生相同的

$$(s', t'),$$

那么对应的量始终是同一个

$$\langle t' | U | s' \rangle.$$

因此只需要计算一次，其结果可以直接复用。缓存只减少重复计算，不改变递推关系本身。所以程序实际上是递推关系的记忆化实现，即

递归 + 记忆化 = 动态规划。

最终，程序计算的量与最初由算符变换得到的非递归表达式完全相同：

$$\langle t | U | s \rangle = \frac{\text{Per}(U_{s,t})}{\sqrt{\prod_i s_i! \prod_j t_j!}}.$$

3.3 递归算法的复杂度

对于固定的目标输出占据态

$$t = (t_1, \dots, t_m), \quad \sum_{j=1}^m t_j = n,$$

算法只会调用满足

$$0 \leq t'_j \leq t_j, \quad j = 1, \dots, m$$

的子问题 $\text{AMP}(s', t')$ 。对于每一个这样的 t' ，递归规则唯一确定与之对应的 s' ，因此每个 t' 至多对应一个不同的递归子问题。由于第 j 个分量 t'_j 有 $t_j + 1$ 种可能取值，不同子问题的数量至多为

$$N(t) \leq \prod_{j=1}^m (t_j + 1) \leq \left(\frac{\sum_{j=1}^m (t_j + 1)}{m} \right)^m = \left(\frac{n + m}{m} \right)^m.$$

利用递归缓存，每个子问题至多计算一次。计算一个子问题时，算法至多需要遍历 m 个模式，故单个子问题的计算开销为 $O(m)$ 。因此，总时间复杂度为

$$T(n, m) = O(mN(t)) = O\left(m \left(\frac{n + m}{m} \right)^m\right).$$

最终得到

$$T(n, m) = O\left(m \left(\frac{n + m}{m} \right)^m\right).$$

在固定 m 的前提下这是关于 n 的多项式复杂度。